

14s nachgedacht

Oui. Voici le **blueprint de base de données Supabase/PostgreSQL** que je prendrais pour construire le produit proprement dès le départ.

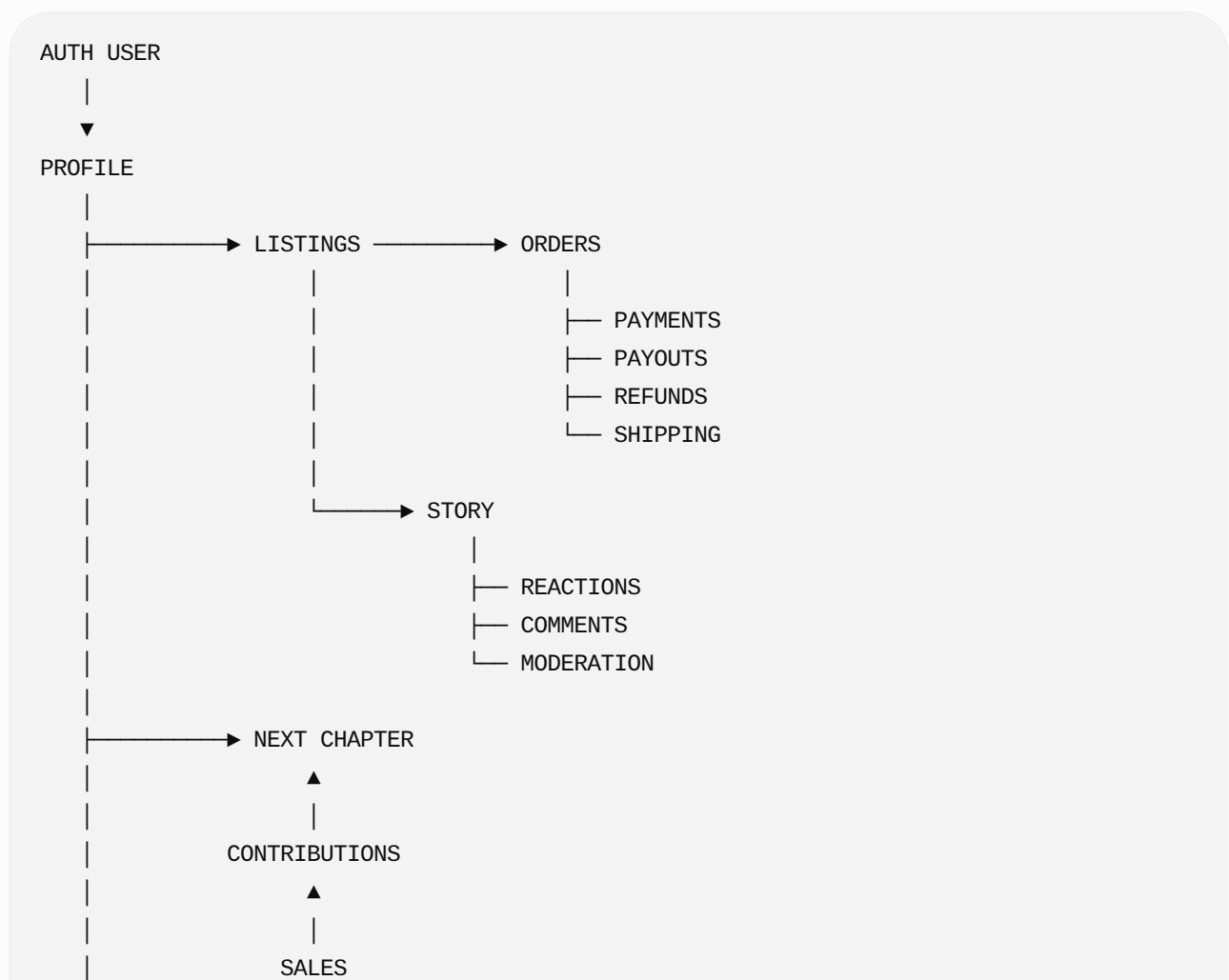
Supabase recommande bien de protéger les tables exposées avec RLS et de ne jamais exposer les clés `service_role` /secret au frontend. `auth.uid()` peut être utilisé directement dans les politiques. [👉 Supabase +1](#)

1. Principes d'architecture

Je poserais six règles dès le début :

1. **Une table = une responsabilité métier claire**
2. `listing ≠ story ≠ order ≠ next_chapter`
3. PostgreSQL reste **source de vérité**
4. Tous les montants monétaires sont stockés en **integer minor units** (`69000` = 690,00 €)
5. RLS activé sur toutes les tables accessibles depuis le client
6. Les opérations sensibles passent par fonctions SQL sécurisées ou Edge Functions

Architecture métier :



2. Les schémas PostgreSQL

Je ne mettrais pas absolument tout dans `public`.

Je séparerais :

```
public
commerce
social
messaging
payments
trust
private
```

Mais pour simplifier les appels Supabase au début, une autre stratégie parfaitement valable consiste à conserver les tables applicatives publiques dans `public` avec RLS strict et les fonctions internes dans des schémas non exposés.

Pour notre MVP, je prendrais :

```
public.*
private.*
audit.*
```

Simple et maintenable.

3. Enums fondamentaux

Je commencerais par des enums PostgreSQL.

SQL

```
create type listing_status as enum (
  'draft',
  'pending_review',
  'active',
  'reserved',
  'sold',
  'paused',
  'rejected',
  'removed'
```

```
);

create type item_condition as enum (
    'new',
    'like_new',
    'very_good',
    'good',
    'fair'
);

create type story_mode as enum (
    'clean_break',
    'little_tea',
    'full_story'
);

create type identity_visibility as enum (
    'public',
    'limited',
    'anonymous'
);

create type moderation_status as enum (
    'pending',
    'approved',
    'needs_changes',
    'rejected',
    'removed'
);

create type order_status as enum (
    'pending_payment',
    'paid',
    'awaiting_shipping',
    'shipped',
    'delivered',
    'completed',
    'cancelled',
    'disputed',
    'refunded'
);

create type offer_status as enum (
    'pending',
    'accepted',
    'declined',
    'withdrawn',
    'expired'
);

create type chapter_status as enum (
    'active',
```

```
'completed',  
'paused',  
'archived'  
);  
  
create type report_status as enum (  
  'open',  
  'reviewing',  
  'resolved',  
  'dismissed'
```

4. profiles

`auth.users` reste géré par Supabase Auth.

On ne duplique pas email/password dans notre table.

SQL

```
create table public.profiles (  
  id uuid primary key references auth.users(id) on delete cascade,  
  
  username text unique,  
  display_name text,  
  
  avatar_path text,  
  bio text,  
  
  country_code char(2),  
  city text,  
  
  is_verified boolean not null default false,  
  is_suspended boolean not null default false,  
  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Je n'exposerais pas :

- adresse exacte ;
- date de naissance ;
- nom légal ;
- téléphone ;
- données KYC ;

dans `profiles`.

5. Données privées du vendeur

SQL

```
create table private.seller_accounts (  
  user_id uuid primary key references auth.users(id) on delete cascade,  
  
  seller_type text not null default 'private',  
  stripe_account_id text unique,  
  
  legal_first_name text,  
  legal_last_name text,  
  
  tax_country char(2),  
  tax_identifiant text,  
  
  kyc_status text,  
  payouts_enabled boolean not null default false,  
  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Cette table n'est jamais directement exposée publiquement.

Et je chifferrais/minimiserais les informations fiscales réellement conservées.

Stripe doit rester la source principale pour les informations KYC lorsque possible.

6. Catégories d'objets

SQL

```
create table public.categories (  
  id bigint generated always as identity primary key,  
  
  parent_id bigint references public.categories(id),  
  
  slug text unique not null,  
  name text not null,  
  
  sort_order integer default 0,  
  is_active boolean not null default true  
);
```

Exemple :

fashion

- └ bags
- └ shoes
- └ clothing

electronics

- └ smartphones
- └ computers
- └ gaming

jewelry

home

wedding

luxury

7. listings

C'est le produit vendu.

Rien d'émotionnel ici.

SQL

```
create table public.listings (  
  id uuid primary key default gen_random_uuid(),  
  
  seller_id uuid not null  
    references public.profiles(id),  
  
  category_id bigint  
    references public.categories(id),  
  
  title text not null,  
  description text,  
  
  brand text,  
  model text,  
  
  condition item_condition not null,  
  
  currency char(3) not null default 'EUR',  
  price_amount integer not null check (price_amount > 0),  
  
  status listing_status not null default 'draft',
```

```
country_code char(2),
city text,

published_at timestamptz,
sold_at timestamptz,

created_at timestamptz not null default now(),
updated_at timestamptz not null default now()
);
```

Exemple :

```
price_amount = 69000
currency = EUR
```

plutôt que :

```
690.00
```

Ça évite énormément de problèmes de précision monétaire.

8. Photos

SQL

```
create table public.listing_images (
  id uuid primary key default gen_random_uuid(),

  listing_id uuid not null
    references public.listings(id)
    on delete cascade,

  storage_path text not null,

  sort_order integer not null default 0,

  moderation_status moderation_status
    not null default 'pending',

  width integer,
  height integer,

  created_at timestamptz not null default now()
);
```

Jamais de URL publique permanente stockée.

On conserve plutôt :

```
bucket/path/file.webp
```

et on génère l'URL appropriée.

9. Les histoires

Voici notre vraie différenciation.

SQL

```
create table public.stories (  
  id uuid primary key default gen_random_uuid(),  
  
  listing_id uuid not null unique  
    references public.listings(id)  
    on delete cascade,  
  
  author_id uuid not null  
    references public.profiles(id),  
  
  mode story_mode not null default 'clean_break',  
  
  visibility identity_visibility  
    not null default 'public',  
  
  headline text,  
  body text,  
  
  moderation_status moderation_status  
    not null default 'pending',  
  
  published_at timestamptz,  
  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Pour commencer :

```
1 listing = maximum 1 story
```

D'où :

SQL

listing_id uuid **unique**

On pourra changer plus tard si nécessaire.

10. Contextes de rupture

Je ne veux surtout pas stocker cela en texte libre.

SQL

```
create table public.relationship_contexts (  
  id smallint generated always as identity primary key,  
  
  slug text unique not null,  
  label text not null,  
  emoji text,  
  
  is_sensitive boolean not null default false,  
  sort_order integer default 0,  
  
  is_active boolean not null default true  
);
```

Exemples :

```
cheated_on  
ghosted  
divorce  
wedding_cancelled  
moving_out  
terrible_gift  
peaceful_goodbye  
what_was_i_thinking  
at_least_they_had_taste  
other
```

Puis :

SQL

```
create table public.story_relationship_contexts (  
  story_id uuid not null  
    references public.stories(id)  
    on delete cascade,  
  
  context_id smallint not null  
    references public.relationship_contexts(id),
```

```
primary key (story_id, context_id)
);
```

Pourquoi N:N ?

Parce qu'une histoire peut être :

```
divorce
+
moving_out
```

ou :

```
wedding_cancelled
+
at_least_they_had_taste
```

11. Réactions

SQL

```
create type reaction_type as enum (
    'dead',
    'red_flag',
    'sending_love',
    'good_for_you',
    'tea',
    'no_way',
    'savage'
);
```

Puis :

SQL

```
create table public.story_reactions (
    story_id uuid not null
        references public.stories(id)
        on delete cascade,

    user_id uuid not null
        references public.profiles(id)
        on delete cascade,

    reaction reaction_type not null,

    created_at timestamptz not null default now(),
```

```
primary key (story_id, user_id)
);
```

Un utilisateur = une réaction active par story.

Il peut la modifier.

12. Commentaires

SQL

```
create table public.comments (
  id uuid primary key default gen_random_uuid(),

  story_id uuid not null
    references public.stories(id)
    on delete cascade,

  author_id uuid not null
    references public.profiles(id),

  parent_comment_id uuid
    references public.comments(id),

  body text not null,

  moderation_status moderation_status
    not null default 'pending',

  created_at timestamptz not null default now(),
  edited_at timestamptz
);
```

On permet ainsi :

```
comment
└─ reply
```

Je limiterais probablement la profondeur à 1 au niveau applicatif.

Pas besoin de devenir Reddit.

13. Saved items

SQL

```
create table public.saved_listings (  
  user_id uuid not null  
    references public.profiles(id)  
    on delete cascade,  
  
  listing_id uuid not null  
    references public.listings(id)  
    on delete cascade,  
  
  created_at timestamptz not null default now(),  
  
  primary key(user_id, listing_id)  
);
```

14. Follow

Je ferais initialement suivre un **profil**, pas une story individuelle.

SQL

```
create table public.follows (  
  follower_id uuid not null  
    references public.profiles(id)  
    on delete cascade,  
  
  followed_id uuid not null  
    references public.profiles(id)  
    on delete cascade,  
  
  created_at timestamptz not null default now(),  
  
  primary key(follower_id, followed_id),  
  
  check (follower_id <> followed_id)  
);
```

15. Next Chapters

SQL

```
create table public.next_chapters (  
  id uuid primary key default gen_random_uuid(),
```

```

owner_id uuid not null
    references public.profiles(id),

title text not null,
description text,

target_amount integer,
currency char(3) not null default 'EUR',

visibility identity_visibility
    not null default 'public',

status chapter_status
    not null default 'active',

completed_at timestamptz,

created_at timestamptz not null default now(),
updated_at timestamptz not null default now()
);

```

Exemple :

```

title:
First solo trip

target_amount:
150000

```

16. Lier les annonces à un Next Chapter

SQL

```

create table public.listing_chapters (
    listing_id uuid primary key
    references public.listings(id)
    on delete cascade,

    chapter_id uuid not null
    references public.next_chapters(id)
    on delete cascade
);

```

Ainsi :

```

ring └─
macbook ┤─> Trip to Bali

```

17. Contributions

C'est important :

on ne modifie jamais directement une progression.

Pas :

```
current_amount = current_amount + 690
```

La progression doit être calculable à partir de transactions réelles.

SQL

```
create table public.chapter_contributions (  
  id uuid primary key default gen_random_uuid(),  
  
  chapter_id uuid not null  
    references public.next_chapters(id),  
  
  order_item_id uuid not null unique,  
  
  amount integer not null check(amount > 0),  
  
  created_at timestamptz not null default now()  
);
```

Puis :

```
progress =  
SUM(valid contributions)  
-  
SUM(refunds)
```

18. Updates du Next Chapter

C'est le futur :

What happened next?

SQL

```
create table public.chapter_updates (  
  id uuid primary key default gen_random_uuid(),  
  
  chapter_id uuid not null  
    references public.next_chapters(id)  
    on delete cascade,  
  
  author_id uuid not null  
    references public.profiles(id),  
  
  body text not null,  
  image_path text,  
  
  moderation_status moderation_status  
    default 'pending',  
  
  published_at timestamptz,  
  
  created_at timestamptz not null default now()  
);
```

19. Offers

SQL

```
create table public.offers (  
  id uuid primary key default gen_random_uuid(),  
  
  listing_id uuid not null  
    references public.listings(id),  
  
  buyer_id uuid not null  
    references public.profiles(id),  
  
  amount integer not null check(amount > 0),  
  
  status offer_status  
    not null default 'pending',  
  
  expires_at timestamptz,  
  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

20. Orders

SQL

```
create table public.orders (  
  id uuid primary key default gen_random_uuid(),  
  
  buyer_id uuid not null  
    references public.profiles(id),  
  
  seller_id uuid not null  
    references public.profiles(id),  
  
  currency char(3) not null,  
  
  subtotal_amount integer not null,  
  buyer_fee_amount integer not null default 0,  
  seller_fee_amount integer not null default 0,  
  shipping_amount integer not null default 0,  
  total_amount integer not null,  
  
  status order_status not null  
    default 'pending_payment',  
  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

21. Order items

Même si une transaction contient initialement un seul objet, je garderais une table séparée.

SQL

```
create table public.order_items (  
  id uuid primary key default gen_random_uuid(),  
  
  order_id uuid not null  
    references public.orders(id),  
  
  listing_id uuid not null unique  
    references public.listings(id),  
  
  price_amount integer not null,  
  
  created_at timestamptz not null default now()  
);
```

Cela permet plus tard :

sans casser toute l'architecture.

22. Payments

SQL

```
create table private.payments (  
  id uuid primary key default gen_random_uuid(),  
  
  order_id uuid not null  
    references public.orders(id),  
  
  provider text not null default 'stripe',  
  
  provider_payment_intent_id text unique,  
  
  amount integer not null,  
  currency char(3) not null,  
  
  status text not null,  
  
  created_at timestamptz not null default now(),  
  updated_at timestamptz not null default now()  
);
```

Le navigateur ne décide jamais :

```
payment.status = paid
```

Ça vient exclusivement d'un webhook Stripe validé.

23. Payouts

SQL

```
create table private.payouts (  
  id uuid primary key default gen_random_uuid(),  
  
  order_id uuid not null  
    references public.orders(id),  
  
  seller_id uuid not null
```

```
references public.profiles(id),

provider_transfer_id text unique,

gross_amount integer not null,
platform_fee_amount integer not null,
net_amount integer not null,

status text not null,

created_at timestamptz not null default now()
);
```

24. Refunds

SQL

```
create table private.refunds (
  id uuid primary key default gen_random_uuid(),

  order_id uuid not null
    references public.orders(id),

  amount integer not null,

  reason text,

  provider_refund_id text unique,

  status text not null,

  created_at timestamptz not null default now()
);
```

25. Conversations

SQL

```
create table public.conversations (
  id uuid primary key default gen_random_uuid(),

  listing_id uuid
    references public.listings(id),

  created_at timestamptz not null default now()
);
```

26. Members

SQL

```
create table public.conversation_members (  
  conversation_id uuid not null  
    references public.conversations(id)  
    on delete cascade,  
  
  user_id uuid not null  
    references public.profiles(id),  
  
  joined_at timestamptz not null default now(),  
  
  primary key(conversation_id, user_id)  
);
```

27. Messages

SQL

```
create table public.messages (  
  id uuid primary key default gen_random_uuid(),  
  
  conversation_id uuid not null  
    references public.conversations(id)  
    on delete cascade,  
  
  sender_id uuid not null  
    references public.profiles(id),  
  
  body text,  
  
  attachment_path text,  
  
  created_at timestamptz not null default now(),  
  edited_at timestamptz,  
  deleted_at timestamptz  
);
```

28. Blocks

Indispensable.

SQL

```
create table public.blocked_users (  
  blocker_id uuid not null  
    references public.profiles(id),  
  
  blocked_id uuid not null  
    references public.profiles(id),  
  
  created_at timestamptz not null default now(),  
  
  primary key(blocker_id, blocked_id)  
);
```

29. Reports

Un report peut viser différents objets.

Je n'aime pas trop les foreign keys polymorphes du genre :

```
target_type  
target_id
```

parce qu'on perd l'intégrité référentielle.

Je préfère :

SQL

```
create table public.reports (  
  id uuid primary key default gen_random_uuid(),  
  
  reporter_id uuid not null  
    references public.profiles(id),  
  
  story_id uuid references public.stories(id),  
  comment_id uuid references public.comments(id),  
  listing_id uuid references public.listings(id),  
  reported_user_id uuid references public.profiles(id),  
  
  reason text not null,  
  details text,  
  
  status report_status not null default 'open',  
  
  created_at timestamptz not null default now(),
```

```
check (  
    num_nonnulls(  
        story_id,  
        comment_id,  
        listing_id,  
        reported_user_id  
    ) = 1  
)  
);
```

Très propre.

Exactement une cible.

30. Moderation cases

SQL

```
create table private.moderation_cases (  
    id uuid primary key default gen_random_uuid(),  
  
    report_id uuid  
        references public.reports(id),  
  
    content_type text not null,  
    content_id uuid not null,  
  
    risk_score numeric(5,4),  
  
    pii_detected boolean default false,  
    harassment_detected boolean default false,  
    doxxing_risk boolean default false,  
    image_risk boolean default false,  
  
    status text not null,  
  
    assigned_to uuid,  
  
    created_at timestamptz not null default now(),  
    resolved_at timestamptz  
);
```

31. Audit logs

Très important pour l'administration.

SQL

```
create table audit.events (  
  id bigint generated always as identity primary key,  
  
  actor_id uuid,  
  
  action text not null,  
  
  entity_type text,  
  entity_id uuid,  
  
  metadata jsonb,  
  
  created_at timestamptz not null default now()  
);
```

Par exemple :

```
moderator_removed_story  
seller_refund_created  
account_suspended  
listing_price_changed
```

32. Notifications

SQL

```
create table public.notifications (  
  id uuid primary key default gen_random_uuid(),  
  
  user_id uuid not null  
    references public.profiles(id),  
  
  type text not null,  
  
  actor_id uuid  
    references public.profiles(id),  
  
  entity_type text,  
  entity_id uuid,  
  
  read_at timestamptz,  
  
  created_at timestamptz not null default now()  
);
```

33. RLS — principe

Sur toutes les tables publiques concernées :

SQL

```
alter table public.listings
enable row level security;
```

Même chose pour :

```
profiles
stories
comments
offers
orders
messages
next_chapters
...
```

Supabase rappelle que RLS fonctionne directement avec Auth et que les politiques sont exécutées pour les accès aux tables ; `auth.uid()` renvoie `null` lorsqu'aucune session n'est présente, donc je préfère toujours des politiques explicitement authentifiées.  Supab...

34. RLS Listings

Tout le monde peut voir une annonce active :

SQL

```
create policy "public can read active listings"
on public.listings
for select
using (
  status = 'active'
  or seller_id = (select auth.uid())
);
```

Création :

SQL

```
create policy "users create own listings"
on public.listings
```

```
for insert
to authenticated
with check (
    seller_id = (select auth.uid())
);
```

Modification :

SQL

```
create policy "seller updates own listings"
on public.listings
for update
to authenticated
using (
    seller_id = (select auth.uid())
)
with check (
    seller_id = (select auth.uid())
);
```

35. RLS Stories

Lecture publique uniquement si :

```
approved
+
published
+
listing active
```

Conceptuellement :

SQL

```
using (
    moderation_status = 'approved'
    or author_id = (select auth.uid())
);
```

36. RLS Messages

C'est une policy extrêmement importante.

Un message est lisible uniquement si :


```
auth.uid()  
∈ conversation_members
```

On peut créer une fonction interne :

SQL

```
security.is_conversation_member(conversation_id)
```

puis :

SQL

```
using (  
  security.is_conversation_member(conversation_id)  
);
```

Les fonctions utilisées par RLS peuvent être placées dans un schéma non exposé et référencées explicitement dans les politiques. Supabase documente ce pattern.  Supab...

37. RLS Orders

Acheteur :

```
buyer_id = auth.uid()
```

OU vendeur :

```
seller_id = auth.uid()
```

Donc :

SQL

```
using (  
  buyer_id = (select auth.uid())  
  or seller_id = (select auth.uid())  
);
```

Mais :

client → pas de UPDATE libre.

On ne veut jamais :

JavaScript

supabase

```
.from("orders")  
.update({ status: "completed" })
```

depuis React.

Les transitions passent par des fonctions métier.

38. State machine

Très important.

Une commande ne peut pas passer arbitrairement :

```
pending_payment  
→ completed
```

Je définirais les transitions :

```
pending_payment  
  |  
  ▼  
  paid  
  |  
  ▼  
  awaiting_shipping  
  |  
  ▼  
  shipped  
  |  
  ▼  
  delivered  
  |  
  ▼  
  completed
```

Branches :

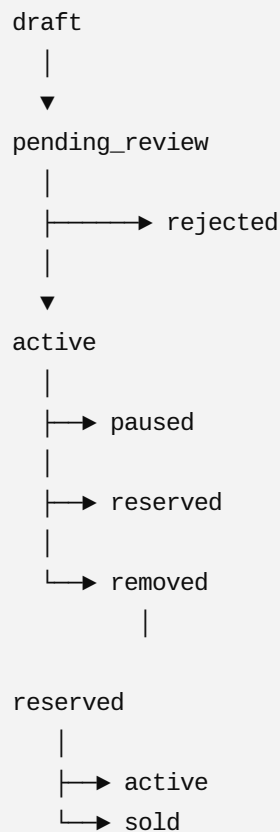
```
pending_payment → cancelled
```

```
paid → cancelled/refunded
```

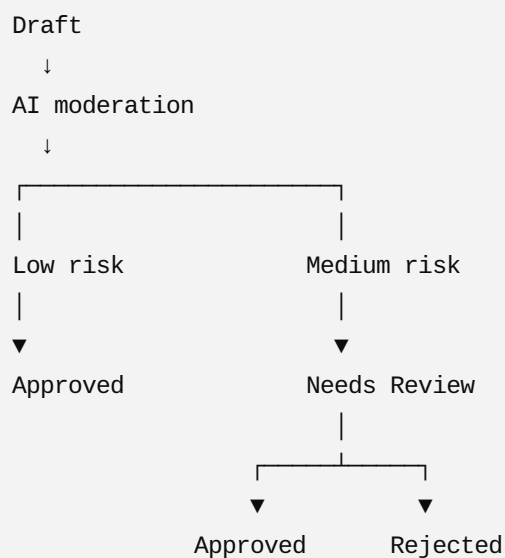
```
shipped → disputed
```

Cette logique appartient au backend.

39. Listing state machine



40. Story moderation workflow



High risk :

Doxxing
Threat
PII
Explicit identity exposure

→ blocked immediately + manual review.

41. Indexes

On indexe les colonnes réellement utilisées dans :

filters,

joins,

RLS,

feed ranking.

Par exemple :

SQL

```
create index listings_seller_idx
on public.listings(seller_id);

create index listings_status_published_idx
on public.listings(status, published_at desc);

create index listings_category_status_idx
on public.listings(category_id, status);

create index stories_author_idx
on public.stories(author_id);

create index stories_published_idx
on public.stories(published_at desc)
where moderation_status = 'approved';

create index messages_conversation_created_idx
on public.messages(conversation_id, created_at desc);

create index orders_buyer_idx
on public.orders(buyer_id, created_at desc);

create index orders_seller_idx
on public.orders(seller_id, created_at desc);
```

Les colonnes utilisées par RLS doivent particulièrement être indexées lorsque le volume augmente, sinon des checks de sécurité très fréquents peuvent devenir coûteux. Supabase souligne également l'importance des indexes et des patterns optimisés pour la performance des politiques.  Supab...

42. Feed

Je ne créerais PAS encore une table :

feed

Le feed peut initialement être dérivé.

Vue :

SQL

```
create view public.story_feed as
select
  ...
from stories s
join listings l
  on l.id = s.listing_id
where
  s.moderation_status = 'approved'
  and l.status = 'active';
```

Puis ranking :

```
score =
recency
+ reactions
+ comments
+ saves
+ personalization
```

Plus tard :

materialized view ou table ranking dédiée.

43. Compteurs

Je ne compterais pas systématiquement :

SQL

```
select count(*)
```

sur plusieurs millions de réactions.

À terme on pourrait avoir :

```
story_stats
```

SQL

```
story_id  
views_count  
reactions_count  
comments_count  
shares_count  
saves_count  
ranking_score
```

Mais pas obligatoire au jour 1.

44. Storage buckets

Je créerais :

```
avatars  
listing-images  
story-media  
chapter-media  
message-attachments  
dispute-evidence
```

dispute-evidence :

privé absolument.

listing-images :

public/signed suivant notre stratégie.

Pour Storage aussi, il faut configurer les policies correctement ; Supabase signale notamment qu'un upload peut échouer si l'INSERT est autorisé mais que la policy SELECT correspondante manque pour le retour des métadonnées.

45. Les migrations

Je voudrais exactement ce genre d'organisation :

```
supabase/  
└─ migrations/  
  
    001_extensions.sql  
    002_enums.sql  
  
    003_profiles.sql  
    004_categories.sql  
  
    005_listings.sql  
    006_listing_images.sql  
  
    007_relationship_contexts.sql  
    008_stories.sql  
    009_reactions_comments.sql  
  
    010_next_chapters.sql  
  
    011_social.sql  
  
    012_conversations.sql  
  
    013_offers.sql  
    014_orders.sql  
  
    015_payments.sql  
  
    016_moderation.sql  
  
    017_indexes.sql  
  
    018_functions.sql  
    019_triggers.sql  
  
    020_rls_profiles.sql  
    021_rls_listings.sql  
    022_rls_stories.sql  
    023_rls_social.sql  
    024_rls_messages.sql  
    025_rls_orders.sql  
  
    026_storage.sql  
  
    027_seed_reference_data.sql
```

Et chaque changement futur :

```
028_add_story_share_stats.sql
029_add_shipping_provider.sql
030_add_offer_expiration.sql
```

Jamais :

je modifie manuellement la prod dans Supabase Dashboard et j'oublie ce que j'ai fait.

46. Fonctions métier

Je créerais certaines opérations atomiques directement côté PostgreSQL.

Par exemple :

```
create_listing
publish_listing
create_offer
accept_offer
reserve_listing
mark_order_paid
mark_listing_sold
create_chapter_contribution
cancel_order
```

Parce que :

```
accept offer
```

peut nécessiter :

```
offer → accepted
listing → reserved
other offers → declined
order → created
```

Ça doit être une transaction atomique.

Soit tout réussit.

Soit rien.

47. Très important : DB Trigger pour les comptes

Quand Supabase crée :

```
auth.users
```

trigger :

```
handle_new_user()
```

→ crée :

```
public.profiles
```

automatiquement.

Donc impossible d'avoir :

```
auth user
```

sans profil correspondant.

48. TypeScript automatiquement dérivé

Après migration Supabase :

```
Bash
```

```
supabase gen types typescript
```

On génère :

```
database.types.ts
```

Donc :

```
TypeScript
```

```
Database["public"]["Tables"]["listings"]["Row"]
```

est dérivé de PostgreSQL.

On ne définit pas une seconde fois manuellement :

TypeScript

```
interface Listing {...}
```

avec le risque qu'elle diverge de la DB.

49. Où utiliser JSONB ?

Très peu.

Oui pour :

```
audit metadata  
moderation model output  
provider raw metadata
```

Non pour :

```
listing  
story  
order  
chapter
```

Je refuse le pattern :

```
SQL  
  
data jsonb
```

pour tout.

Ça tue :

types,

foreign keys,

indexes,

data integrity,

analytics.

50. Schéma MVP final

Au lancement, notre cœur serait donc approximativement :

AUTH

- └─ auth.users

IDENTITY

- └─ profiles
- └─ seller_accounts

MARKETPLACE

- └─ categories
- └─ listings
- └─ listing_images
- └─ saved_listings
- └─ offers

STORY

- └─ stories
- └─ relationship_contexts
- └─ story_relationship_contexts
- └─ story_reactions
- └─ comments

NEXT CHAPTER

- └─ next_chapters
- └─ listing_chapters
- └─ chapter_contributions
- └─ chapter_updates

SOCIAL

- └─ follows
- └─ blocked_users
- └─ notifications

MESSAGING

- └─ conversations
- └─ conversation_members
- └─ messages

TRANSACTIONS

- └─ orders
- └─ order_items
- └─ payments
- └─ payouts
- └─ refunds

TRUST & SAFETY

- └─ reports
- └─ moderation_cases

Soit environ **30 tables**, mais chacune simple et compréhensible.

Je préfère largement cela à 8 tables surchargées.

51. Une décision importante : on ne connecte pas encore Stripe au début

Je séparerais même notre développement en deux versions fonctionnelles.

MVP Alpha

```
signup
→ create listing
→ story
→ feed
→ reactions
→ save
→ comments
→ next chapter
→ share
```

Pas encore d'argent réel.

Objectif :

valider la mécanique sociale.

MVP Beta

Puis :

```
offers
→ checkout
→ Stripe Connect
→ KYC
→ order
→ shipping
→ payout
→ contribution
→ Past Sold ✓
```

Objectif :

valider la marketplace.

Ça réduit énormément le risque.

52. Et cela nous donne maintenant l'architecture applicative

```
apps/web
|
├─ app/
|   ├─ (marketing)/
|   ├─ (auth)/
|   ├─ feed/
|   ├─ discover/
|   ├─ sell/
|   ├─ listing/[id]/
|   ├─ story/[id]/
|   ├─ chapter/[id]/
|   ├─ inbox/
|   ├─ orders/
|   └─ profile/[username]/
|
├─ features/
|   ├─ auth/
|   ├─ listings/
|   ├─ stories/
|   ├─ feed/
|   ├─ reactions/
|   ├─ comments/
|   ├─ chapters/
|   ├─ offers/
|   ├─ checkout/
|   ├─ messaging/
|   └─ moderation/
|
└─ components/
```

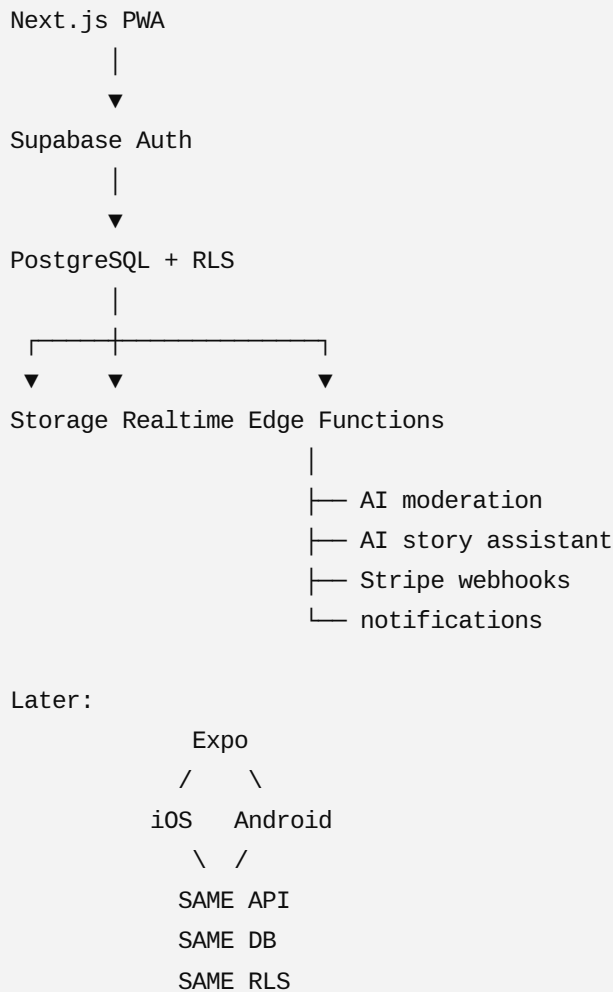
Et non :

```
components/
  ListingCard.tsx
  ListingForm.tsx
  Story.tsx
  Message.tsx
  ...
```

avec toute la logique dispersée.

Chaque domaine métier possède son propre module.

Ce que je considère maintenant comme notre architecture V1 de référence



Et surtout : **je ne toucherais presque plus à cette séparation conceptuelle**. On pourra ajouter des colonnes et fonctionnalités, mais `listing`, `story`, `next_chapter`, `order`, `payment` et `moderation` doivent rester des domaines séparés.

La prochaine étape utile n'est même plus de réfléchir théoriquement : on peut désormais passer à **la Phase 0 puis Phase 1 réelle**, avec structure exacte du repository, variables `.env`, installation Next.js/Supabase, conventions Git, design system, premières migrations SQL et ordre exact des tickets à développer jusqu'à obtenir le premier prototype PWA fonctionnel.